

Ex04

Answers & Manual

Paul Tröger, Fynn Gohlke, Tom Rodewald, Jan Büchele

May 19, 2026

1 Answers

1.1 Non-private logistic regression

The accuracy of our non-private logistic regression is around 0.94.

1.2 Gradient Descent with Renyi Differential Privacy

In each gradient descent iteration we spend $\epsilon_{\text{bar}} / \text{iterations}$ privacy budget by adding noise to the average gradient. Through sequential composition this adds up per iteration, so after all iterations are done we have a total cost of ϵ_{bar} . RDPGD has lower accuracy (0.7 vs 0.94) than the non-private logistic regression, as we add noise to protect privacy. This, however, makes the gradient less precise, so there is a privacy-utility trade-off.

1.3 Differentially Private Stochastic Gradient Descent

The results of our evaluation can be found in fig. 1. First, we can see that iterations have by far the largest impact on accuracy, with 500 being the best overall. Learning rate also seems to have a high impact, however, if a learning rate above 0.1 is chosen, the accuracy fluctuates a lot, thus leading to inconsistent results. A learning rate of 0.1 seems like a good compromise. Both batch size and epsilon seem to have an overall smaller impact, with $\epsilon=1$ and $\text{batch_size}=32$ seeming good. We can recommend using a large number of iterations, with 500 performing the best overall. A learning rate of 0.1 with $\epsilon=1$ and $\text{batch_size}=32$ are reasonable choices.

1.4 Performance evaluation

Our comparisons can be found in fig. 2. The plot shows that increasing the number of iterations improves the utility of both algorithms, but for DPSGD the effect is much stronger, while RDPGD only improves very minimally across iterations. Additionally, the runtime increases very significantly for RDPGD, while DPSGD barely has any increase. For DPSGD, batch size has a smaller and less significant effect on accuracy, with the best result in being at batch size 16. Runtime increases very slightly with larger batch sizes. The main reason for the runtime difference is that RDPGD always uses the full training data for gradient descent. DPSGD, on the other hand, only uses a batch of the training data, so each iteration is much cheaper and the total runtime is much lower.

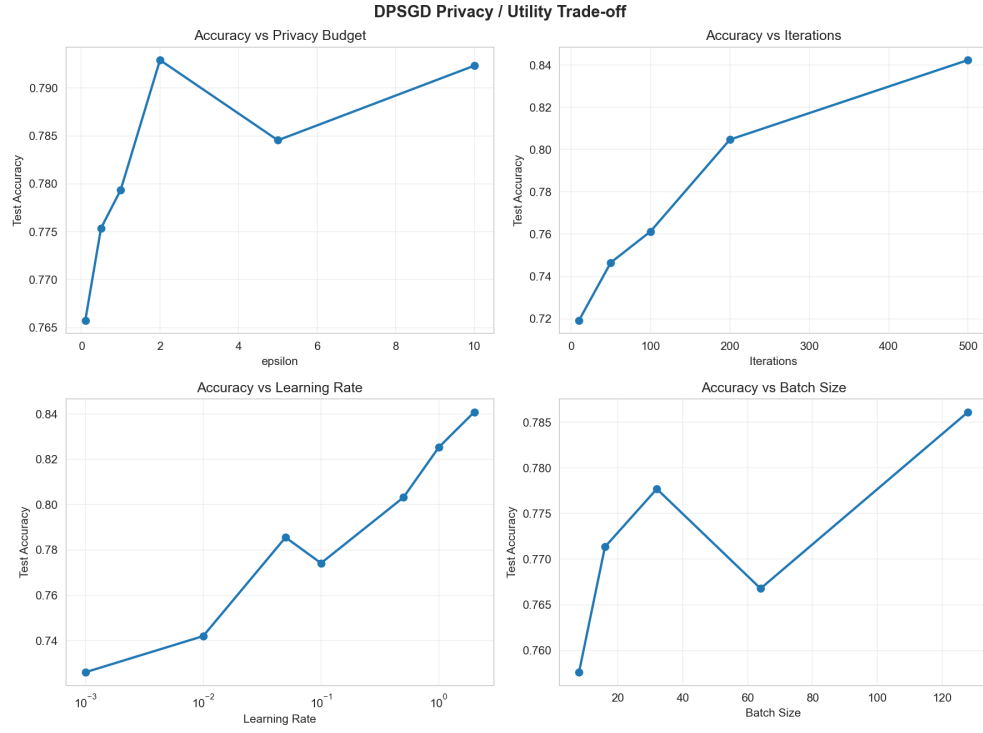


Figure 1: Utility Trade-off

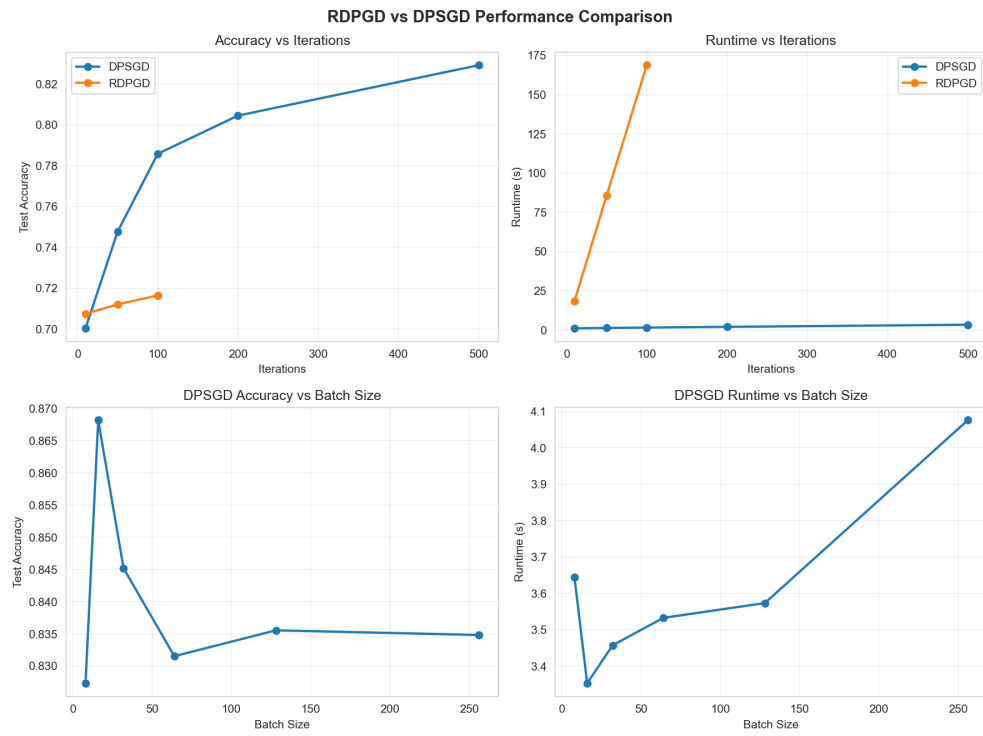


Figure 2: Performance Comparison

2 Manual

2.1 Running the code

All source code is self contained within a singular file called "e3.py".

Running the application can be achieved using *python3 e3.py*.

There are also python tests defined within the application. These can be run using *pytest -v e3.py*.

The flag "-v" is not mandatory, but improves clarity on what actually happened.

2.2 Test Cases

2.2.1 Task 1 Tests

For task one we test the `prepare_data` function. Checking for the right shape and if we removed NaN values.

2.2.2 Other Tests

We have not specified extra tests for Tasks 3-4 since the exercise itself was to test and evaluate the functions.